Ders 10 Çalışma Özeti

Ders 10 Çalışma Özeti

Genel Konular

- Feature Envy (özellik kıskançlığı) kod kusuru (ekrandan, 00:10:00)
  - Bir metot, ait olduğu sınıfın üyelerinden çok başka sınıfların üyelerine erişiyorsa bu duruma özellik kıskançlığı denir. Buradaki "özellik" (feature) daha çok üye alan anlamındadır.
  - Daha dar bir biçimi: bir üye alana kendi sınıfının metotlarından çok başka sınıfların nesneleri erişiyorsa bu da bir tür Feature Envy'dir.
  - "Çok" sıfatı, o koddaki ortalama farklı sınıf üyesi erişim sayısının üzerinde bir değer olarak tanımlanır. Hoca eşiğin %50 bile fazla görünebileceğini, %25-%20'nin de bir uyarı sayılabileceğini; eşiğin projenin kod kalite ölçütlerinde belirlenebileceğini söyledi.
  - Kök neden: sorumlulukların doğru atanmamasıdır. Bağlaşım nedeniyle her sınıf başka sınıflara erişecektir; hiçbir sınıf yalnız kendi üyeleriyle çalışmaz (öyle olsa uyum ilkesi bozulurdu). Sorun, erişimin ağırlıkla dış kaynaklara yönelmesidir.
  - Slayta göre Strategy ve Visitor kalıpları bir özellik kıskançlığı durumu olarak yorumlanmamalıdır.
  - Çözüm: Move Method; kıskançlık metodun yalnız bir kısmındaysa Extract Method ile o kısım çıkarılıp kıskanılan üyelerin sınıfına taşınır. Üye alanlar genelde doğru yerdedir, çünkü varlığın bilgilerini keşfetmek sorumluluk atamaktan kolaydır; hata çoğunlukla metotlardadır.
- Bir tasarımın ardışık düzenlemelerle değiştirilmesi: video kiralama örneği
  - Kaynak Fowler'ın *Refactoring* kitabının 1. bölümüdür. `Customer` sıfır ya da daha çok `Rental` tutar, her `Rental` bir `Movie` ile ilişkilidir, her filmin fiyat kodu vardır (`REGULAR = 0`, `NEW_RELEASE = 1`, `CHILDRENS = 2`; ekrandan, 01:30:00). `statement()` makbuzu `String` olarak döndürür.
  - Her adım ayrı bir pakette (`fowler_00` … `fowler_10`, `fowler_10_alt`) tutuldu; her adımdan sonra birim testleri (yalnız paket adı değiştirilerek kopyalanan `TestAll`, `TestCustomer`, `TestMovie`) çalıştırıldı ve Eclipse'in karşılaştırma (compare) aracıyla önceki sürüm gösterildi.
  - Kusurların odağı uzun `statement()` metodudur; en uzun parçası olan switch-case bloğundan başlanır.
- Adım 1: Extract Method, `amountFor` (ekrandan, 00:20:00)
  - Switch-case bloğu `private double amountFor(Rental each)` metoduna çıkarılır; döngüde `thisAmount = amountFor(each);` kalır. Metot iç çalışmaya yönelik olduğundan `private` yapılır.
  - Eclipse'in Refactor > Extract Method aracı: yerel değişkenler seçimin içinde kalınca gereksiz yere `thisAmount`'u parametre yaptı. Bir öğrencinin önerisiyle `double thisAmount = 0;` satırı seçimin içine alınınca araç doğru metodu üretti.
- Adım 2: Rename (ekrandan, 00:40:00)
  - `each` → `aRental`, `thisAmount` → `result`. Bir kod kusurunu gidermez ama anlaşılabilirliği artırdığı için bir refactoring eylemidir.
- Adım 3: Move Method, `getCharge`
  - `amountFor`, `Customer`'ın hiçbir üyesine erişmez; yalnız kira kaydının ve filmin alanlarına getter'larla erişir: Feature Envy. Metot `Rental` sınıfına `getCharge()` adıyla taşınır; parametreye gerek kalmaz. Hoca `getCharge`'ın `public` olmasında sakınca görmedi.

  - `Rental.getCharge()` için ek bir birim testi yazmak zorunlu değildir: black box test sürdürülebilir; white box testi kod kapsamasını (code coverage) artırır ama test bütçesini büyütür.
- Adım 4: Replace Temp with Query (ekrandan, 00:49:10)
  - `thisAmount` bir kez atanıp değişmediği için kaldırılır; kullanıldığı iki yerde `aRental.getCharge()` çağrılır. `totalAmount`, `frequentRenterPoints` (kümülatif toplamlar) ve `result` (dönüş değeri) bu aşamada kaldırılamaz.
  - Bir müşterinin kiraladığı film sayısı küçük olduğundan ek çağrıların başarıma etkisi hissedilmez.
- Adım 5: Bonus için Extract + Move Method (ekrandan, 01:00:00, 01:17:00)
  - `frequentRenterPoints` → `bonus` olarak yeniden adlandırılır. Bonus hesabı doğrudan `Rental.getBonus()` metoduna çıkarılıp taşınır; geçici değişken yerine `return 2; / return 1;` kullanılır.
- Adım 6: `getTotalCharge()` ve `getTotalBonus()` (ekrandan, 01:10:00)
  - Kalan iki geçici değişken için sorgu metotları `Customer`'a eklenir (Replace Temp with Query + Extract Method). `statement()` kısalır. Sorgu metotlarındaki `result` adının `totalCharge/bonus` olması daha anlaşılır olabilir. Alt bilgi (footer) satırları da Extract Method ile çıkarılabilir.
- Adım 7 ve 8: `getCharge` ve `getBonus` `Movie`'ye taşınır (ekrandan, 01:17:00)
  - Switch'in anahtarı filmin fiyat kodudur; iş mantığı ağırlıkla filme aittir. `Movie` yalnız bir veri sınıfıydı (data class). `Rental.getCharge()` tek satıra iner: `return _movie.getCharge(_daysRented);`. `getBonus` için de aynı yapılır.
  - Kazanç: yeni bir film türü (ör. Blu-ray) eklemek yalnız `Movie` sınıfını değiştirir; önceki sürümde hem `Movie` hem `Rental` değişirdi (Shotgun Surgery).
- Adım 9: Customer-Movie doğrudan bağlaşımının kaldırılması (ekrandan, 01:22:40)
  - 8. adımdan sonra `Customer` ile `Movie` arasındaki tek doğrudan ilişki `getTitle` mesajıdır. `Rental`'a `getMovieTitle()` eklenir; `Customer` yalnız `Rental` ile, `Rental` yalnız `Movie` ile konuşur (katmanlı iletişim). Dolaylı ilişki tasarım gereği sürer.
- Adım 10: Replace Type Code with Subclasses (ekrandan, 01:30:30)
  - `Movie` soyut olur (`getCharge(int)` ve `getBonus(int)` soyut); `MovieChildrens`, `MovieNewRelease`, `MovieRegular` alt sınıfları her case'in davranışını taşır; switch kalkar.
  - Bedel: `Movie(String title, int priceCode)` kurucusu ve `getPriceCode()` kalktığı için `TestMovie` kopyalanamaz; dış arayüz değişir. Veri tabanı yapısı da etkilenirse geriye doğru uyumluluk kaybolur. Bu adımın yapılıp yapılmayacağı (ör. 2.2 sürümü mü, 3.0 ana sürümü mü) teknik olduğu kadar mali ve pazarlama boyutu olan bir karardır.
- Alternatif: Replace Type Code with State/Strategy (ekrandan, 01:35:10)
  - `Movie` ve tip kodu kalır; soyut `Price` sınıfı (`getCharge(int)`, `getPriceCode()` soyut; `getBonus(int)` somut, 1 döndürür) ve `PriceChildrens`, `PriceNewRelease`, `PriceRegular` alt sınıfları eklenir. Yalnız `PriceNewRelease` `getBonus`'u override eder.
  - `getBonus`'ta varsayılan davranış olduğu için arayüz yerine soyut sınıf kullanıldı. `Price` nesnesi oluşturulurken bir switch kalır ama hesaplama mantığı içermez. Kazanç: birim testleri değişmez.
- Birinci ara sınav sorularının tartışması: tasarım kalıbı seçimi (ekrandan, 01:51:35, 02:00:00, 02:24:40)
  - Soru 1 (ülkeye göre değişen adres, tarih, telefon sınıfları): uygun kalıplar Abstract Factory, Factory Method, Builder, Bridge, Strategy, Visitor, Template Method; ülke kuralları için Singleton da düşünülebilir ama kopya kod üretmemelidir. Kısmen uygun: Decorator, Prototype, Adapter, MVC. Uygun değil: Flyweight, kişi verisi için Singleton.
  - Soru 2 (departman, alt departman, çalışan hiyerarşisi ve raporlama): Composite (asıl yanıt) ve Visitor uygun; Prototype, Iterator, Chain of Responsibility kısmen uygun.

Hocanın Özellikle Vurguladığı Kısımlar

- Switch-case ya da if-else zinciri yazdığınızı görüyorsanız büyük olasılıkla orada bir kalıtım hiyerarşisi kurup çok biçimliliği kullanmanız gerekiyordu.
- Değişiklikler küçük adımlarla yapılır; bir adım hata çıkarırsa kolayca geri alınır. Her adımdan sonra birim testleri çalıştırılır; testler değiştirilmez.
- Bir refactoring adımı yeni kod kusurları doğurabilir; bu yüzden her adımdan sonra en son

değiştirilen kod yeniden incelenir (ör. `amountFor`'daki Feature Envy).
- Eclipse'in otomatik Extract Method'u karmaşık durumlarda gereksiz parametre aktarabilir; çıktısını denetlemek gerekir.
- Tip kodlarını kaldırmanın en ucuz zamanı yazılım sahaya çıkmadan öncesidir; sonradan yapılan arayüz değişikliği geriye doğru uyumluluğu bozar.
- Kalıp sorularında Facade, Adapter ve Mediator genelde ana yanıt değildir; ana kalıbın yanında ikincil bir gereksinim için düşünülür. Adapter yalnız uymayan metot imzalarını uyduran bir ara katmandır.
- MVC "her şeye oturan" bir kalıp olduğu için tek başına güçlü bir yanıt sayılmaz.

Kısa Tekrar Notları

- Feature Envy: metot başka sınıfların üyelerine kendi sınıfınınkinden çok erişir → Move Method, gerekirse önce Extract Method.
- Sıra: Extract Method (`amountFor`) → Rename → Move Method (`Rental.getCharge`) → Replace Temp with Query (`thisAmount`) → Extract + Move (`Rental.getBonus`) → `getTotalCharge/getTotalBonus` → `Movie.getCharge(int)` → `Movie.getBonus(int)` → `Rental.getMovieTitle()` → Replace Type Code with Subclasses ya da State/Strategy.
- Rename de bir refactoring eylemidir.
- Black box: iç yapı bilinmez, girdi-çıktı karşılaştırılır. White box: dallanmalar ve döngüler üzerinden kod kapsaması hedeflenir.
- Alt sınıflarla çözüm switch'i tümden kaldırır ama arayüzü ve testleri bozar; Strategy/State çözümü tip kodunu korur, testleri bozmaz.
- Composite: hiyerarşik parça-bütün; Visitor: var olan yapıda dolaşıp yeni işlemler (raporlar) eklemek; Iterator yalnız dolaşır; Chain of Responsibility'de yanıtı zincirdeki tek nesne verir.
- Flyweight için durum bilgisi içsel ve dışsal olarak ikiye ayrılabilmelidir.

Detaylı Açıklamalar

- Feature Envy'nin ölçülmesi: Hoca "çok" ifadesini göreli tanımlar. Bir metodun başka sınıfların üyelerine yaptığı erişim, koddaki ortalama farklı sınıf üyesi erişim sayısının üzerindeyse kusurdan şüphelenilir. Eşik projenin kalite ölçütlerinde tanımlanabilir; bağlaşım arttıkça eşiği daha sıkı tutmak anlamlı olabilir. Feature Envy bir belirtidir; kök neden sorumlulukların yanlış atanmasıdır.

- Adım 1'in kodu (ekrandan, 00:20:00):

```
private double amountFor( Rental each ) {
    double thisAmount = 0;
    //determine amounts for each line
    switch (each.getMovie().getPriceCode()) {
    case Movie.REGULAR:
        thisAmount += 2;
        if (each.getDaysRented() > 2)
            thisAmount += (each.getDaysRented() - 2) * 1.5;
        break;
    case Movie.NEW_RELEASE:
        thisAmount += each.getDaysRented() * 3;
        break;
    case Movie.CHILDRENS:
        thisAmount += 1.5;
        if (each.getDaysRented() > 3)
            thisAmount += (each.getDaysRented() - 3) * 1.5;
        break;
    }
    return thisAmount;
}
```

- Adım 4'ün döngüsü (ekrandan, 00:49:10): `thisAmount` kalktıktan sonra döngüde `String.valueOf(aRental.getCharge())` ve `totalAmount += aRental.getCharge();` kul-

lanılır. Hoca her kira kaydı için `getCharge`'ın iki kez çağrılmasının, kiralanan film sayısı küçük olduğundan başarımı hissedilir ölçüde etkilemeyeceğini söyledi.

- Adım 7'den sonra `Rental` (ekrandan, 01:17:00):

```
public double getCharge( ) {
    return _movie.getCharge( _daysRented );
}
public int getBonus( ) {
    if ((getMovie().getPriceCode() == Movie.NEW_RELEASE)
            && getDaysRented() > 1)
        return 2;
    else
        return 1;
}
```

  - `getBonus` 8. adımda aynı biçimde `Movie.getBonus(int daysRented)` metoduna taşınır; `Rental` yalnız kira gün sayısını filme iletir.

- Adım 10 soyut sınıfı (ekrandan, 01:30:30):

```
public abstract class Movie {
    private String _title;
    public Movie(String title ) {
        _title = title;
    }
    public String getTitle (){
        return _title;
    }
    public abstract double getCharge( int daysRented );
    public abstract int getBonus( int daysRented );
}
```

- Alternatif çözümün soyut stratejisi (ekrandan, 01:35:10):

```
public abstract class Price {
    public abstract double getCharge( int daysRented );
    public abstract int getPriceCode( );
    public int getBonus( int daysRented ) {
        return 1;
    }
}
```

- Sıra diyagramı: Hoca her adımda bir sıra diyagramı da gösterdi; amaç sıra diyagramını öğretmek değil, 9. adımda `Customer`'dan `Movie`'ye giden doğrudan mesajın nasıl kaldırıldığını göstermekti.
- Sınav sorusu tartışmasında Decorator'ın zayıflığı: en küçük ortak adres yapısını çekirdek alıp ülkeye göre eklemeler yapmak düşünülebilir; ama daha az alanı olan yeni bir ülke eklenince çekirdek değişir ve var olan ülkenin (ör. Türkiye) kodu da değişmek zorunda kalır. Prototype'ta ise bütün alanları toplayan "şişkin" bir prototipin bazı üyeleri belirli ülkelerde atıl kalır.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.